



UNICAMP

**Relatório do Trabalho 1: Procedimentos básicos
em imagens digitais**

Relatório desenvolvido a partir da aplicação
dos procedimentos em imagens digitais
apresentados nas aulas de MC920 Introdução
ao Processamento Digital de Imagem

Prof. Dr. Hélio Pedrini

Letícia Silva Brito
240314

1. Introdução	2
2. Materiais	2
2.1. Requerimentos	2
2.2. Organização das entradas e das saídas	2
2.2.1. Entrada	3
2.2.2. Saída	3
2.3. Organização do Código	3
2.3.1. main.py	3
2.3.2. utils.py	3
2.3.3. pencil_sketch.py	4
2.3.4. brightness_adjustment.py	4
2.3.5. mosaic.py	5
2.3.6. filters.py	6
2.3.6.1. Efeito de fotos antigas (1.4)	6
2.3.6.2. Transformação de imagens coloridas (1.5)	7
2.3.6.3. Filtragem de imagens (1.10)	8
2.3.7. bit_planes.py	8
2.3.8. combine_images.py	9
2.3.9. intensity.py	9
2.3.10. quantization.py	11

1. Introdução

Este trabalho constituiu da aplicação de conceitos ensinados em classe em imagens digitais. Ao todo são 10 tipos de procedimentos que foram realizados.

Os códigos que realizaram tais operações são encontrados no repositório Github: <https://github.com/Asteria101/MC920/tree/main/T-01>.

2. Materiais

Esta seção abordará o que é necessário para a execução e organização do código, de modo a tratar, além do código, a organização das entradas e saídas.

Fragmentou-se o código principal em diversos arquivos os quais contém as operações requeridas para imagens digitais, abaixo detalhou-se mais sobre cada um deles ao passo que analisa cada procedimento ou item do enunciado.

2.1. Requerimentos

Este projeto foi desenvolvido na linguagem de programação *Python* na versão 3.12.3, utilizando as seguintes bibliotecas (instaladas em um ambiente virtual, através do comando `pip install ...`), que possibilitaram a criação de algoritmos eficazes para trabalhar com imagens digitais:

Tabela 1: bibliotecas utilizadas e suas respectivas versões

Biblioteca	Versão
Scikit Image	0.25.2
NumPy	2.2.4
Matplotlib	3.10.1

A biblioteca `Scikit Image` foi utilizada para realizar algumas operações mais específicas com as imagens, porém algumas básicas como ler a imagem em um vetor `NumPy`. Assim, a biblioteca `NumPy` serve para realizar operações de Álgebra Linear e Geometria Analítica com as imagens digitais. Por fim, `Matplotlib` é usada para visualizar e salvar as imagens na máquina.

2.2. Organização das entradas e das saídas

Abaixo detalhou-se como foi feita a ordenação e separação das imagens de entrada e saída de modo a facilitar a chamada e visualização de tais arquivos no diretório.

2.2.1. Entrada

Utilizou-se 8 (oito) arquivos `.png` que foram usados de entrada se encontram no diretório `in-images`, nele há imagens fonte tanto monocromáticas quanto coloridas RGB. As imagens provêm do [diretório](#) de imagens fornecidas pelo Professor Hélio Pedrini.

Como alguns itens exigem que a imagem seja monocromática, fez-se uma função presente no arquivo `utils.py` (item 2.3.2.) que é responsável por identificar se ela é RGB e torná-la monocromática. Tomou-se essa decisão para observar como cada operação reagiria com diferentes imagens.

2.2.2. Saída

A quantidade de saídas varia de item para item. Contudo, encontram-se no diretório `out-images` em diretórios com a numeração dos itens de 1 a 10. O nome de cada arquivo de saída segue a regra geral `nº do problema + . + nome da imagem fonte`, podendo sofrer alterações conforme for mais adequado para cada item. Organizou-se a nomenclatura dos arquivos para conseguir alcançar uma otimização na busca visual por arquivos.

2.3. Organização do Código

A boa estruturação de um programa gera uma maior eficiência e facilidade de manutenção, assim, optou-se por dividir o código em alguns arquivos. Abaixo, há a explicação de cada um contendo a análise do item proposto.

2.3.1. `main.py`

Contém a chamada de todos os arquivos para realização de suas funções assim como o armazenamento das imagens em seus respectivos lugares.

2.3.2. `utils.py`

Este arquivo contém as importações de bibliotecas necessárias para o funcionamento dos códigos, assim como uma função básica `isRGB()` para verificar se a imagem (de tipo `ndarray`) trata-se de uma imagem colorida ou não.

2.3.3. pencil_sketch.py

O objetivo deste item é transformar uma imagem colorida RGB em uma imagem com efeito de esboço de lápis. Transformou-se a imagem colorida em tons de cinzas (i), depois inverteu-se a intensidade dos pixels da imagem cinza (ii) - de modo que onde era mais claro ficou mais escuro e vice-versa, isto é, obteve-se o “negativo” da imagem. Aplicou-se assim um filtro gaussiano 21x21 (iii) e então realçou-se a imagem com base nos tons de cinza, normalizando os pixels entre 0 e 1 (iv).

i (Imagem 1)



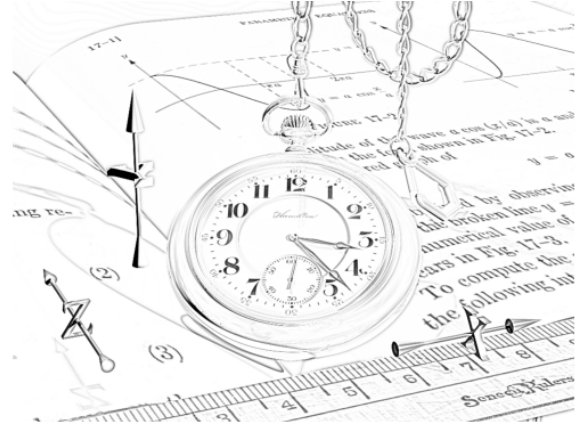
ii (Imagem 2)



iii (Imagem 3)



iv (Imagem 4)



2.3.4. brightness_adjustment.py

O objetivo deste item é poder alterar o brilho de uma imagem através de uma correção gamma. Para isto, o código deve trabalhar com uma imagem monocromática, normalizando-o para um intervalo de 0 e 1, antes e depois da aplicação da equação $B = A^{(1/\gamma)}$. Obteve-se as seguintes imagens para os seguintes valores de γ : 0.05, 0.2, 0.5, 1, 1.5, 2.5 e 5.

Imagem 5

Imagem 6

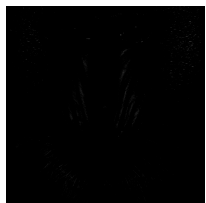
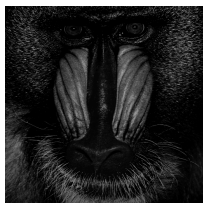
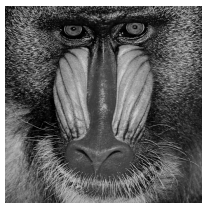
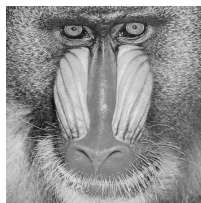
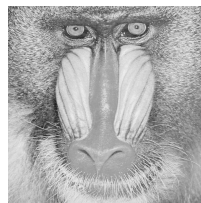
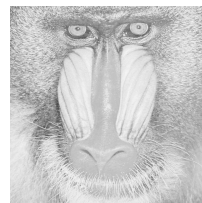
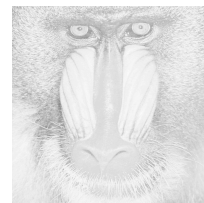
Imagem 7

Imagem 8

Imagem 9

Imagem 10

Imagem 11

 $\gamma = 0.05$  $\gamma = 0.2$  $\gamma = 0.5$  $\gamma = 1$  $\gamma = 1.5$  $\gamma = 2.5$  $\gamma = 5$

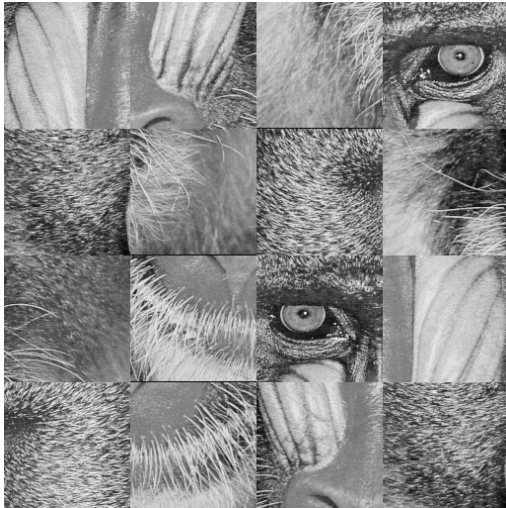
Analisando as imagens acima concluiu-se que quando gamma é igual a 1 a imagem final (Imagem 8) equivale a imagem fonte. Ademais, para valores entre 0 e 1, excluindo o próprio zero, percebeu-se que a imagem tende a escurecer, já para valores maiores que 1 o brilho tende a aumentar.

2.3.5. mosaic.py

O objetivo deste item é dividir a imagem em 16 blocos, transformando a imagem em um mosaico de 4 por 4 blocos. Assim, trabalhou-se com imagens monocromáticas e modificou o formato das imagens, através da função `reshape`, que recebeu os parâmetros:

- `n_blocks`
- `src_img.shape[0]//n_blocks`
- `src_img.shape[1]//n_blocks`

O primeiro parâmetro define a quantidade de blocos na vertical e na horizontal. O segundo e o terceiro definem a altura e largura do bloco. Depois, criou-se uma imagem de mesmas dimensões que a imagem fonte e realocou-se os blocos na ordem desejada. Por fim, rearranjou-se a imagem para as dimensões originais. E assim, obteve-se exemplos dos resultados abaixo:



baboon_colored.png mosaic (Imagem 12)



watch.png mosaic (Imagem 13)

2.3.6. filters.py

Neste arquivo há a aplicação dos seguintes itens propostos: 1.4, 1.5 e 1.10. Para discutir sobre eles, dividir-se-á esta seção em três, para que se aborde da melhor forma cada problema.

2.3.6.1. Efeito de fotos antigas (1.4)

Este problema recebe uma imagem colorida em formato RGB e então realiza a aplicação do filtro desejado sobre cada pixel por meio de uma multiplicação de matrizes. Normalizando os novos valores num intervalo de 0 a 255. Dessa maneira obtém-se as seguintes imagens.

watch.png: imagem fonte (Imagem 14)



watch.png com filtro (Imagem 15)



peppers.png: imagem fonte (Imagem 16)

peppers.png com filtro (Imagem 17)



2.3.6.2. Transformação de imagens coloridas (1.5)

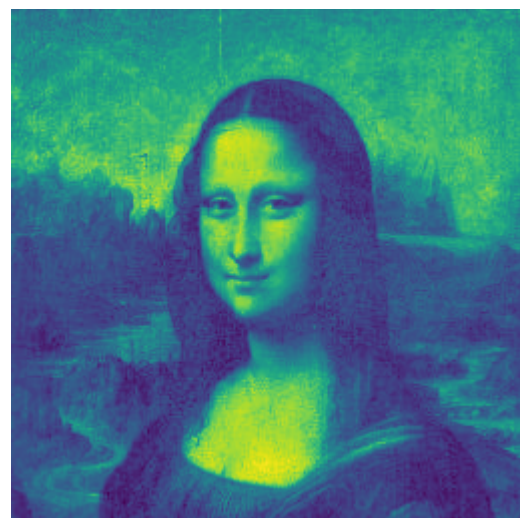
Este item possui dois subitens **a)** e **b)**. O primeiro é extremamente similar ao item anterior visto que ao trabalhar nos canais propostos (R, G e B) podemos interpretar o sistema proposto como uma matriz cujos valores são os índices do sistema. Assim, nota-se que o subitem **a)** equivale ao item anterior.

Já o subitem **b)**, o procedimento apresentado é diferente, visto que, ao receber uma imagem fonte colorida ele a transforma em uma imagem monocromática - algo que pode ser verificado através da função `isRGB()` presente no código do . Como as imagens abaixo:

monalisa.png: imagem fonte (Imagem 18)

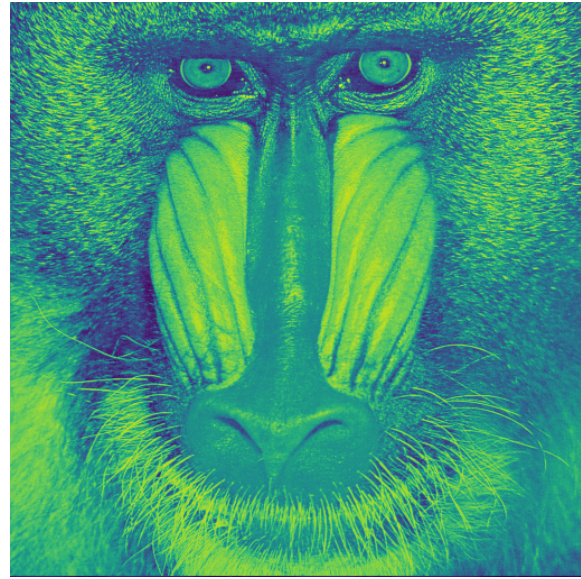


monalisa.png com filtro (Imagem 19)



baboon_colored.png: imagem fonte (Imagem 20)

baboon_colored.png com filtro (Imagem 21)



Nota-se que optou-se por não deixar as imagens finais em tons de cinza, a fim de ressaltar que a imagem não precisa ser cinza para ser monocromática. Ademais, o armazenamento das imagens de saída foram divididas em duas pastas dentro do diretório T-01/out-images/5/: a e b.

2.3.6.3. Filtragem de imagens (1.10)

Neste item aplicou-se vários filtros na imagem house.png.

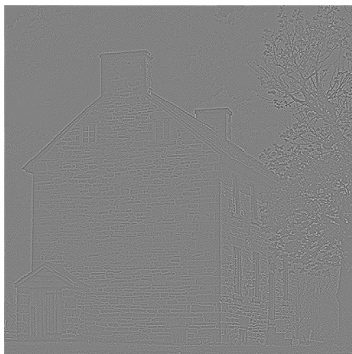


Imagem 46: filtro h1

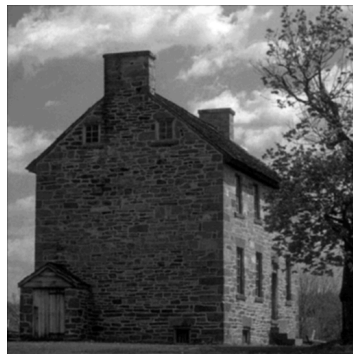


Imagem 47: filtro h2

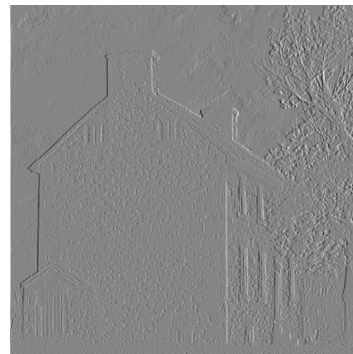


Imagem 48: filtro h3

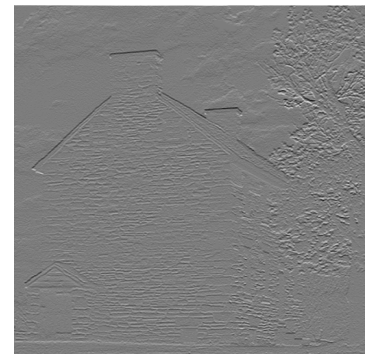


Imagem 49: filtro h4

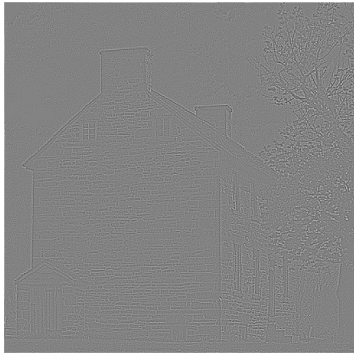


Imagem 50: filtro h5

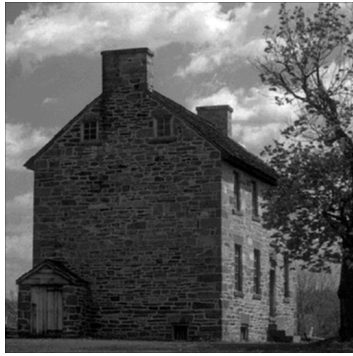


Imagem 51: filtro h6

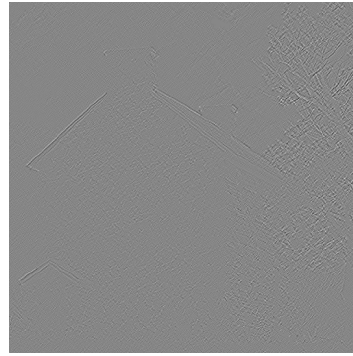


Imagem 52: filtro h7

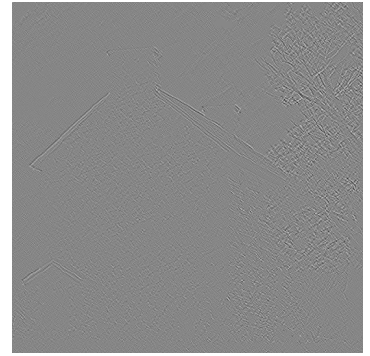


Imagem 53: filtro h8



Imagem 54: filtro h9



Imagem 55: filtro h10



Imagem 56: filtro h11

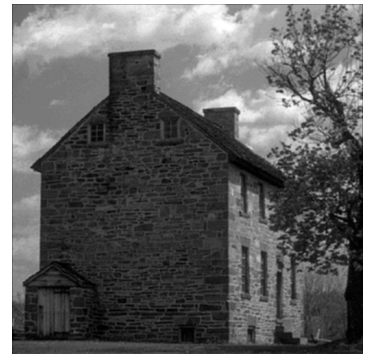


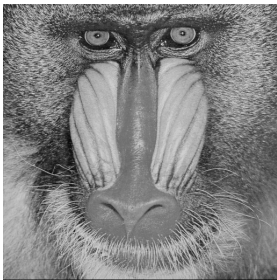
Imagem 57: filtro h12

- Filtro h1: Destaque de bordas internas e externas. Isto é, os contornos mais marcantes;
- Filtro h2: Desfoque na imagem;
- Filtro h3: Foco no destaque das bordas verticais;
- Filtro h4: Foco no destaque das bordas horizontais;
- Filtro h12: Junção dos filtros h3 e h4 de modo que anulou o efeito;
- Filtro h5: extremamente similar ao h1, também serve para destacar bordas/contornos;
- Filtro h6: uma média responsável por desfocar ou suavizar os pixels;
- Filtro h7: Destaque nos contornos, porém com certa presença de desfoque;
- Filtro h8: Muito similar ao h7, porém com um efeito diferente no destaque de bordas;
- Filtro h9: Borramento da imagem, alto desfoque, muito similar ao h6. Porém suaviza muito a imagem através de uma média muito pequena;
- Filtro h10: Há certo destaque nas bordas porém o efeito principal é o aspecto de assimilação dos cinzas;
- Filtro h11: Outro filtro que também pode ser no destaque de bordas e contornos, porém mantendo certa profundidade.

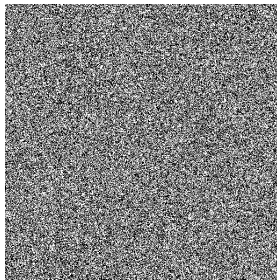
2.3.7. bit_planes.py

Este arquivo contém a aplicação do item 1.6 do enunciado do trabalho cujo objetivo é dividir a imagens em alguns planos de n bits. Para isto, usou-se operações bitwise `>>` e `&`: o primeiro é responsável por fazer a exponenciação necessária para o polinômio e, o segundo vai tirar apenas o bit menos significativo, isolando o bit i do pixel. Assim, obteve-se:

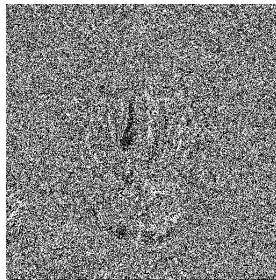
imagem fonte
(Imagem 22)



plano de bit 0
(Imagem 23)



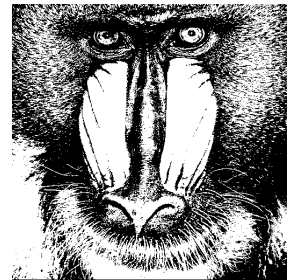
plano de bit 3
(Imagem 24)



plano de bit 5
(Imagem 25)



plano de bit 7
(Imagem 26)

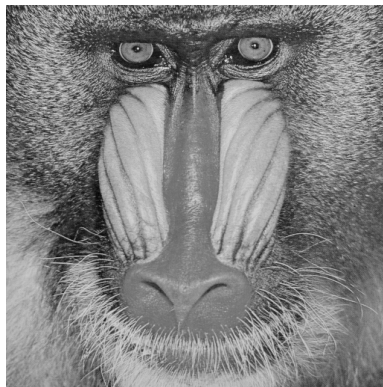


2.3.8. combine_images.py

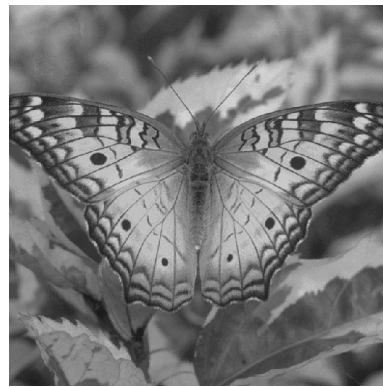
Utilizou-se duas imagens monocromáticas de mesmo tamanho, e assim multiplicou-se os pesos com suas respectivas imagens e somou-se.

```
combined_image = (weight1 * image1 + weight2 *  
                  image2)
```

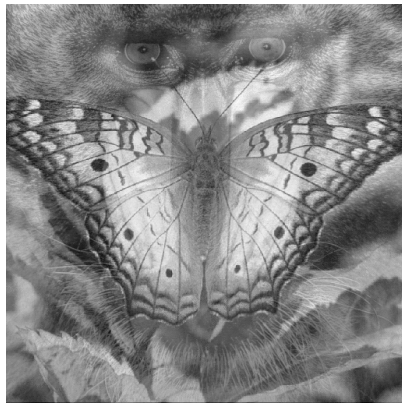
Assim, obteve-se as imagens abaixo:



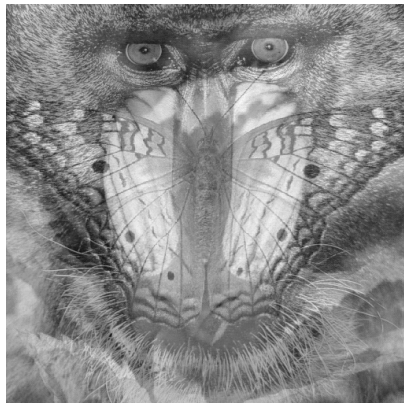
baboon_monochromatic.png: image1
(Image 27)



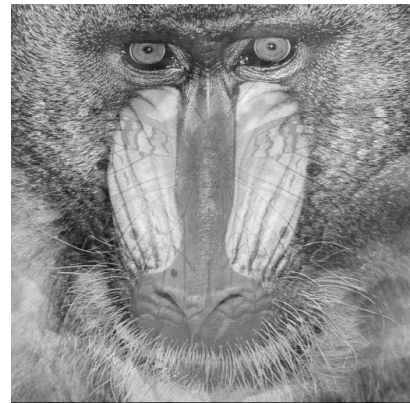
butterfly.png: image2
(Image 28)



weight1: 0.3, weight2: 0.7
(Image 29)



weight1: 0.5, weight2: 0.5
(Image 30)



weight1: 0.7, weight2: 0.3
(Image 31)

2.3.9. intensity.py

Este item dedica-se a entender e a desenvolver os procedimentos de transformação de intensidade, propostos no enunciado do item 1.8, em uma imagem fonte monocromática (Imagem 32). Como são 5 transformações, destrinchar-se-á sobre elas uma a uma abaixo:

a) Negativo da imagem

Esta operação não é muito complicada e chegou a um resultado promissor negando a `src_img`. Isto é, inverteu-se os níveis de cinza para que resultasse na imagem negativa como é possível analisar na Imagem 33 abaixo.

b) Aumentar o contraste

Esta operação é responsável por mudar o intervalo dos valores de cinza de $[0, 255]$ para $[100, 200]$, o que implica na seguinte compreensão: os níveis mais baixos de cinza (<100) ficarão mais escuros e os níveis de cinza ≥ 200 devem ficar levemente mais claros. Assim, é possível notar uma alteração no contraste, deixando a imagem (Imagem 34) mais escura.

c) Inversão das linhas pares

Esta operação pode ser realizada através de um simples *slicing*, mas é preciso ter uma imagem que seja igual a imagem fonte e então, deve-se acessar cada linha “pulando de 2 em 2” tanto para a imagem final quanto para a fonte, de modo que a imagem final deve receber a linha fonte invertida, isto é, `inverted_even_lines_image[::2] = src_img[::2, ::-1]`.

Pode-se conferir o resultado na Imagem 35.

d) Reflexão de Linhas

Para realizar esta operação copiou-se a imagem fonte na variável `reflected_lines_image` e alocou-se a metade superior na

variável `top_half = src_img[:height // 2]`, onde `height = src_img.shape[0]`. Desta maneira, para que a imagem final tivesse o efeito procurado `reflected_lines_image[height // 2:] = top_half[::-1]`. O resultado desta operação pode ser visualizado na Imagem 36.

e) Espelhamento vertical

Para realizar esta operação basta inverter cada linha com um comando de *splicing*. Da seguinte maneira `vertical_mirror_image = src_img[::-1]`. Esta operação assemelha-se muito à rotação 180° da imagem fonte. Sendo possível observar o resultado na Imagem 37.



Imagem 32: imagem fonte



Imagem 33: negativo de city.png (a)

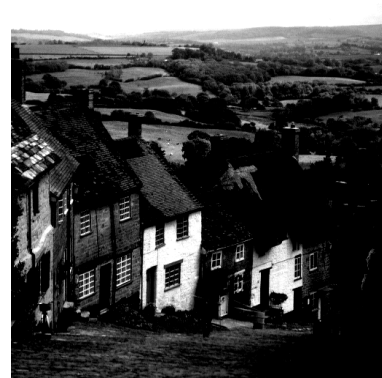


Imagem 34: aumento de contraste de city.png (b)

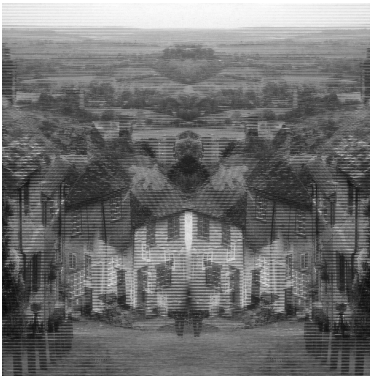


Imagem 35: inversão das linhas pares de city.png (c)

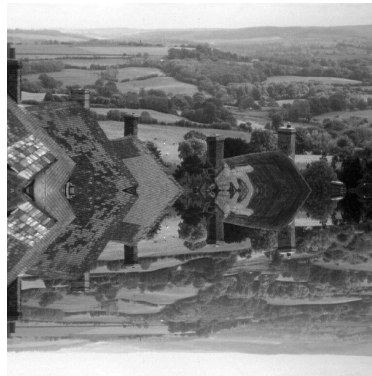


Imagem 36: reflexão das linhas da metade superior de city.png (d)



Imagem 37: espelhamento vertical de city.png (e)

2.3.10. quantization.py

Este item aborda quantização, que se refere à quantidade de níveis de cinza em uma imagem monocromática. Para conseguir imagens com diferentes quantidades de níveis de cinza pode-se realizar o seguinte:

```
level = 2 ** (8 - bit_level)
```

```
quantized_image = (src_img // level) * level
```

A função usada para tal aplicação tem seguinte protótipo `quantization(src_img: np.ndarray, bit_level: int)`, onde `src_img` equivale a imagem fonte e `bit_level` o expoente da base 2 que será usado no cálculo da quantidade de níveis de cinza que devem ser deduzidos da imagem original. De modo que as entradas de 1 a 8 são válidas para `bit_level`, uma vez que o cálculo dos níveis é dado por `level`. Assim, se `bit_level` é igual a 1, então `level` é igual a 128 o que significa que todos os pixels cujo valor for menor que 128 assumirão o valor 0 e os de valor maior ou igual a 128 assumirão o valor 128.

Desta maneira, pode-se obter 8 imagens com diferentes níveis de cinza como é possível visualizar abaixo.

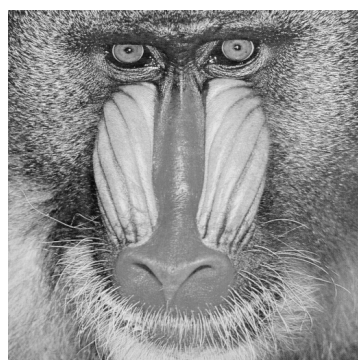


Imagem 38: 256 níveis

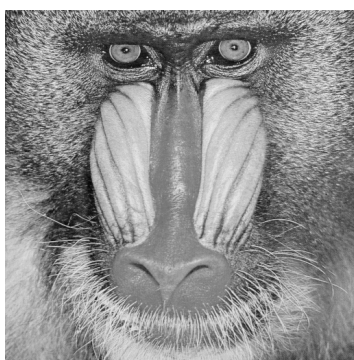


Imagem 39: 128 níveis

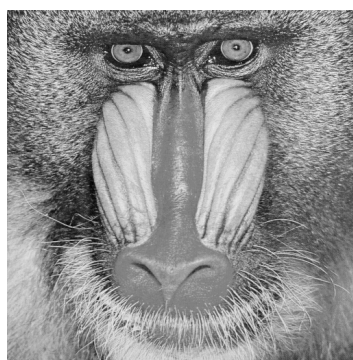


Imagem 40: 64 níveis

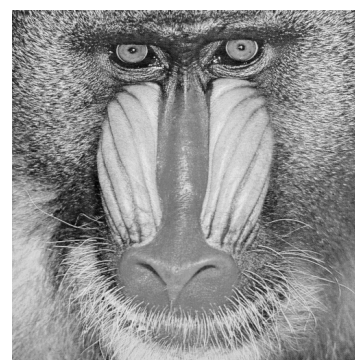


Imagem 41: 32 níveis

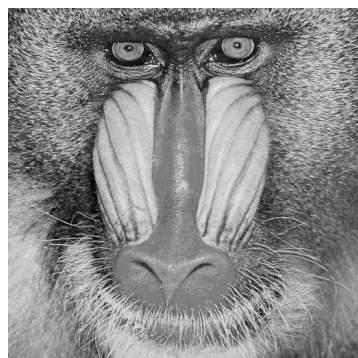


Imagem 42: 16 níveis

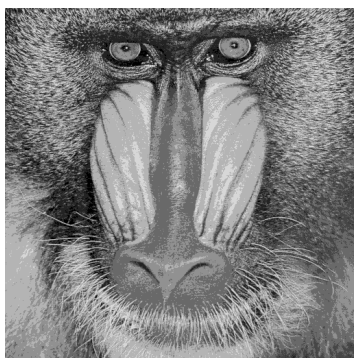


Imagem 43: 8 níveis

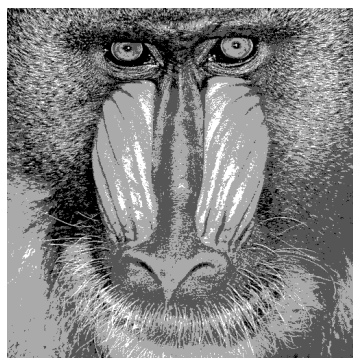


Imagem 44: 4 níveis

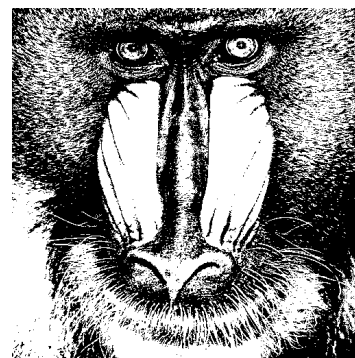


Imagem 45: 2 níveis

Observou-se que as imagens 38, 39, 40, 41 e 42 são muito similares, de modo que conclui-se que é mais proveitoso, em termos de armazenamento, ter imagens com menos níveis de cinza, pois possuem qualidade muito similar à imagem original com 256 níveis (Imagem 38).